Testing Workshop

Name: Fernanda Tarazona

Unit and integration tests were applied with pytest, coverage measurement with pytest-cov and security and code quality analysis with SonarQube on a pipeline that reads clients from a CSV, cleans them and loads them into SQLite. As a complement to SonarQube, Bandit, another static analysis tool (SAST), was used.

Tests: 30 in total (units with parametrize and pytest.raises, and CSV → SQLite integration). All pass (Figures 1 and 2).

Coverage: 100% of the etl package, including transform.py, exceeding the goal of 80% (Figure 2).

Integration: idempotency (run twice leaves 2 rows, not 4) and reconciliation between source and destination were tested.

Invalid email in the CSV: "all or nothing" was decided: the pipeline throws ValueError and does not load any rows, so as not to leave partial data in the table.

Findings of the analysis with the faulty code

A version of reportes.py with intentional defects was analyzed. Integration tests also revealed them: with that code 4 tests failed (SQL injection, eval, and error handling), which shows that the tests and static analysis complement each other.

Defect	SonarQube	Bandit	Correction applied
Password written in the code (DB_PASSWORD = "admin123")	He did not mark it	B105, Low Severity (CWE-259)	Read from an environment variable with os.environ.get
SQL query armed with f-string	He did not mark it	B608, Medium Severity (CWE-89)	Query parameterized with "?"
Using eval on external data	He did not mark it	B307, Medium Severity (CWE-78)	Rules (index, operator, value) with the operator module, without eval
except empty with pass	Yes: Maintainability, severity High (line 20)	B110, Low Severity (CWE-703)	Specific exceptions (TypeError, IndexError) and ValueError is relaunched
Unused "row" parameter	Yes: Maintainability, Severity Medium (line 13)	Not applicable	The rewritten function uses all of its parameters

SonarQube (Sonar way core profile) reported 2 issues, both maintainability issues (Figure 4). It did not mark the password, eval, or SQL query in this configuration. Bandit detected all four security and error handling flaws (Figures 5 and 6).

Result after corrections

- All 30 tests pass and coverage is 100% with code corrected (Figure 7).
- Bandit termina con "No issues identified" (Figura 7).
- SonarQube shows the Quality Gate in Passed, with 0 security, reliability, and maintainability issues, and 100% coverage (Figure 3).

Conclusion

No single tool found everything. SonarQube Community, with the profile used, detected only 2 of the 5 problems, while Bandit identified the 4 security and error handling problems, and pytest tests exposed SQL injection and the use of eval when failing. That's why it's convenient to combine testing, static analysis, and human review.

It was also seen that high coverage does not guarantee the absence of defects: with vulnerable code, coverage was already high and the risk was still there, because assertions are what verify behavior. Executing all this in each commit (shift-left) allows correction before reaching production, where a defect in a transformation is replicated in each load, report and model that consumes that data.

Appendix: evidence

Testing & Coverage

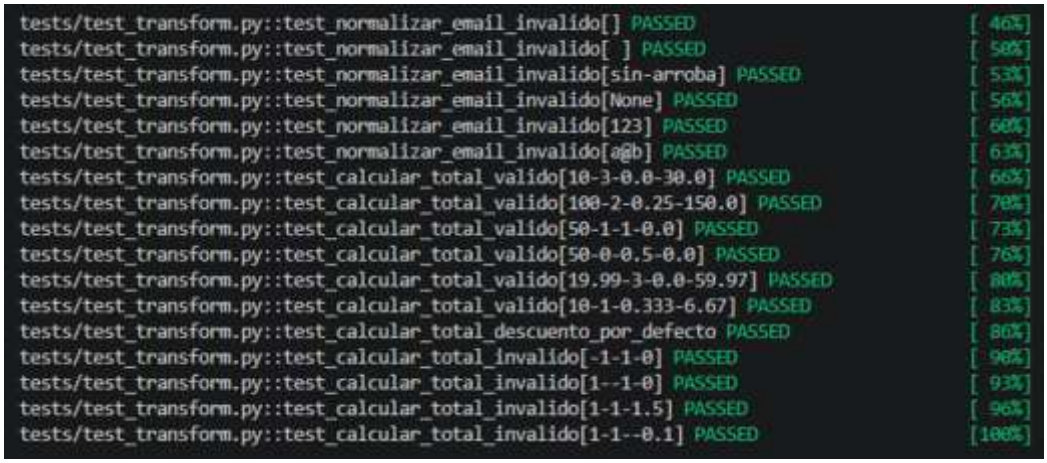


Figure 1. *Green transform.py unit tests (PASSED, 100%).*

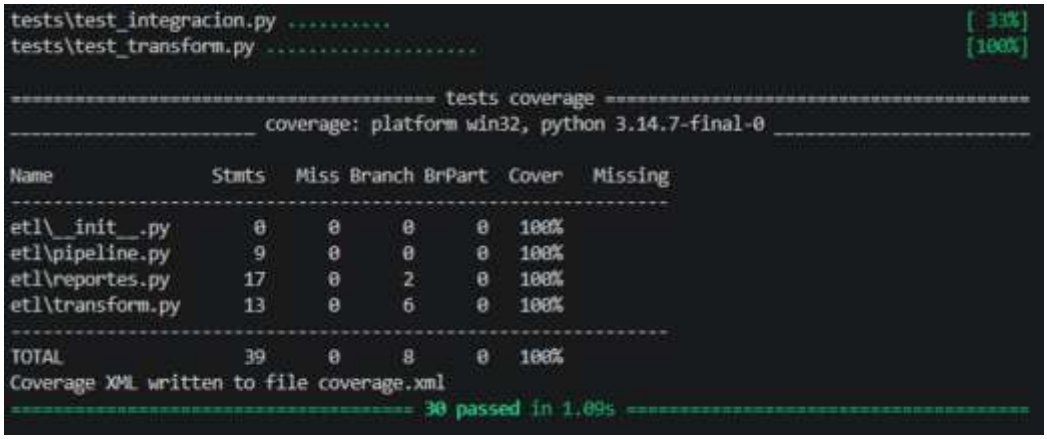


Figure 2. *30 passed tests and 100% coverage across all ETL modules.*

SonarQube



Figure 3. *SonarQube with the code fixed: Quality Gate Passed, 0 issues, and 100% coverage.*

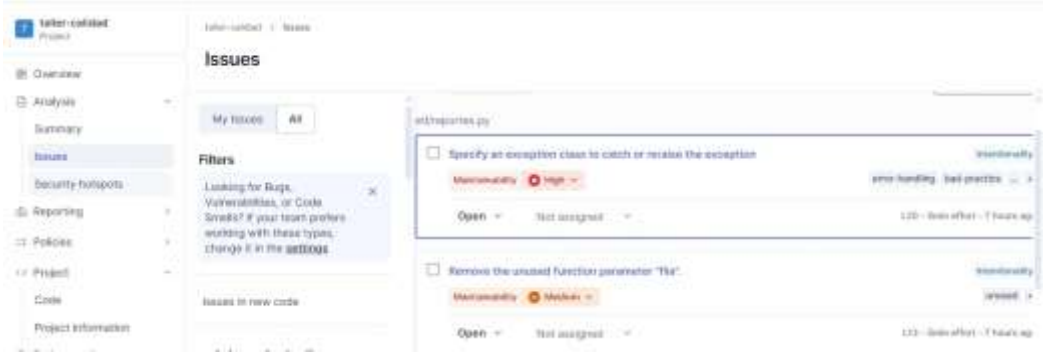


Figure 4. *SonarQube before fixing: 2 maintainability issues in etl/reportes.py.*

Bandit with faulty code

```
Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'admin123'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: etl\reportes.py:5:14
4
5       DB_PASSWORD = "admin123"
6


--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Low
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: etl\reportes.py:9:17
8       def buscar_cliente(con, email):
9           consulta = f"SELECT * FROM clientes WHERE email = '{email}'"
10          return con.execute(consulta).fetchall()

--------------------------------------------------
```

Figure 5. *Bandit: password written in the code (B105) and SQL injection (B608).*

```
--------------------------------------------------
>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
   Severity: Medium   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b307-eval
   Location: etl\reportes.py:14:11
13      def evaluar_regla(expresion, fila):
14          return eval(expresion)
15


--------------------------------------------------
>> Issue: [B110:try_except_pass] Try, Except, Pass detected.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b110_try_except_pass.html
   Location: etl\reportes.py:20:4
19              return sum(f[1] for f in filas)
20          except:
21              pass
```

Figure 6. *Bandit: use of eval (B307) and try/except/pass (B110).*

Fixed code

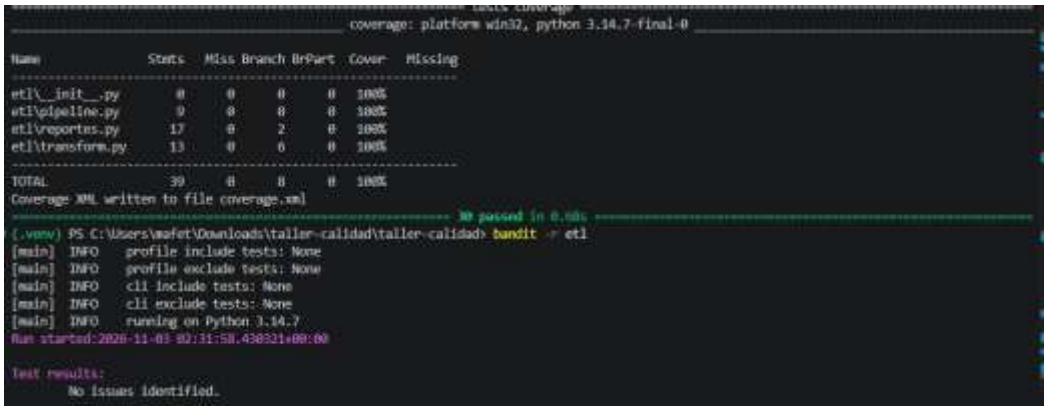


Figure 7. *With code fixed: 30 tests passed, 100% coverage, and Bandit without issues.*